

Toward Excellence in Systems Engineering

Naohiko Hirano

September 28, 1994

English Edition

Prepared from the original Japanese document. Historical terminology and structure are retained as closely as possible.

Contents

1. Aiming for Excellence in Systems Engineering
2. Strategies for Successful Systems Engineering
3. Software Engineering Environment
4. Methodology
5. Tools
6. Reengineering
7. Reusability
8. Human Resources
9. Management

1. Aiming for Excellence in Systems Engineering

What Systems Development Should Aim For

- Development periods are too long in relation to business needs.
- Systems do not always match management and business strategies.
- Development depends heavily on manual work and therefore requires significant effort.
- System documentation can be difficult to understand.
- Documentation and system information can be difficult to maintain.
- Programs and systems can remain difficult to understand even after development.

Approach

The document addresses these issues through systems engineering, methodology, tools, reengineering, reusability, human resources, and management.

2. Strategies for Successful Systems Engineering

Area	Purpose / Need	Main Contents
Methodology	Establish a systematic basis for system development and effective use of CASE.	Methods for system planning and construction; guidance for effective system development and CASE use.
Tools	Improve accuracy and productivity while reducing development time and cost.	Integrated CASE and other tools supporting the methodology.
Reengineering	Improve understanding and maintenance of existing systems.	Analysis, restructuring, reverse engineering, and related techniques.
Reusability	Improve productivity and quality and reduce development time and cost.	Reuse of software and other assets managed through repositories and libraries.
Human Resources	Enable methodology and tools to be used effectively.	Appropriate roles, skills, education, and training.
Management	Coordinate new methods, tools, and people effectively.	Management mechanisms, evaluation, responsibilities, and appropriate planning.

3. Software Engineering Environment

Integrated CASE Environment

The original diagram presents an integrated software engineering environment centered on CASE tools and a repository. System information, analysis and design information, programs, documentation, and existing business systems are connected within the environment.

Key terms in the original figure include:

- Integrated CASE
- Repository
- System analyst
- System engineer
- Existing business system
- Program
- Documentation
- Reusable information

The original figure is retained as a historical reference in the reference section of this edition.

4. Methodology

Methodology

The document describes methodology as a systematic way of applying techniques to system development and of connecting requirements, analysis, design, and implementation.

The document emphasizes that methods should provide a sound basis for system development and should be used consistently with appropriate tools and skills.

A quotation attributed to C. A. Hoare of Oxford University is used to stress the importance of sound foundations when applying software-engineering techniques.

Methodological areas mentioned in the original include information engineering, structured approaches, object-oriented approaches, and analysis and design techniques.

5. Tools

CASE tools

The document presents CASE tools as a means of supporting system development activities that would otherwise require extensive manual work.

Expected benefits include improved productivity, shorter development time, better consistency, and improved handling of system information and documentation.

The historical examples and terminology in the original are retained because they represent the technology and thinking of 1994.

6. Reengineering

Reengineering

The document describes reengineering as an approach for analyzing and transforming existing systems while making effective use of information already contained in those systems.

Related activities include analysis, restructuring, reverse engineering, and migration and transformation.

The purpose is to support maintenance, understanding, improvement, and continued use of existing system assets.

7. Reusability

Reusability

The document identifies reuse as a means of improving productivity, quality, development speed, and cost performance by making effective use of accumulated development assets.

Potential reusable assets include programs, design information, documentation, analysis and design models, methods, and other information stored in repositories or libraries.

8. Human Resources

Human Resources

Systems engineering depends on people who can use the methodology and tools effectively and who understand their roles within the development process.

Roles mentioned include project management, system analysis and engineering, CASE-related work, programming, and other specialist activities.

The document also emphasizes education and training so that new methods and tools can be applied effectively.

9. Management

Management

Management is presented as the activity that coordinates methodology, tools, people, objectives, responsibilities, and execution.

- Understand organizational needs as a whole.
- Clarify objectives and plans.
- Evaluate systems from an overall perspective.
- Create an appropriate working environment.
- Clarify responsibilities and roles.
- Evaluate work and clarify accountability.
- Select and assign appropriate personnel.

Reference: Original Japanese Visual Material

The following pages reproduce the supplied original Japanese scans containing diagrams, tables, screenshots, and other visual material. They are included so that the English text can be checked against the historical source.

方法論

一連のタスクを組み合わせて使用し、作業者を結論に導く
為のガイドである。

必要性

一貫した考えや規準及び明確なダイアグラムがなかった為に
混乱が生じてきた。

Oxford大学 C. A. Hoare教授談

「あいまいな基礎の上に、ソフトウェア工学の作業規律を適用すると、きっと悲惨な結果をまねくであろう。それは
、化学の基礎を燃素の上に置いたり、天文学の基礎を地球が平板だという仮説の上に置くのと同様である。」

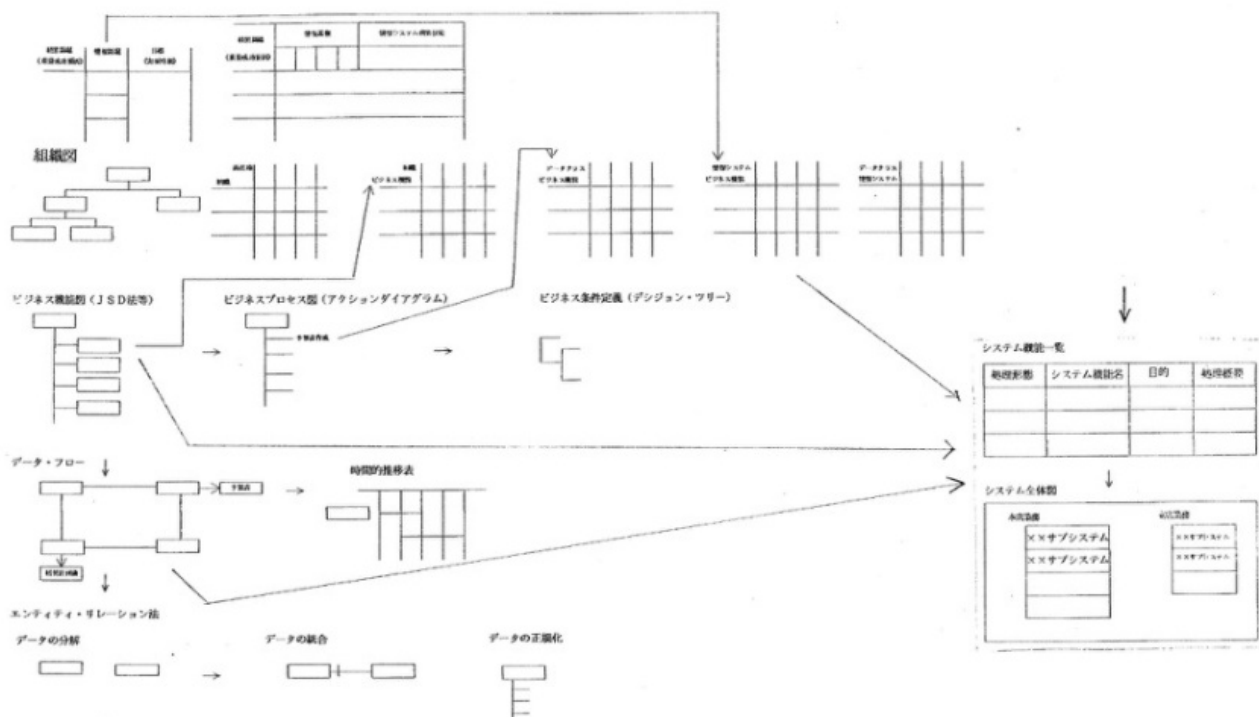
方法論の種類

戦略

- ・ 5つの競争要因、3つの基本戦略、価値連鎖分析
- ・ 戦略的グリッド
- ・ 重要成功要因分析
- ・ S W O T分析等

設計（注 IEは戦略から構築までの流れでできている）

- ・ I E（インフォメーション・エンジニアリング）
- ・ S S A D M
- ・ オブジェクト指向設計
- ・ J S D法等



ツール

CASEツールはコンピュータを使用して、システムの企画から構築、及びプログラム作成迄を支援するシステムである。

必要性

- ・手書きのダイアグラムでは数多くのエラー、非一貫性、抜けが残っていた。
- ・設計書は膨大なページになり、手作業に関連のチェックを行うのは困難
- ・プログラム作成も非生産性であった。

効果

- ・システムの開発を短期間で行うことができる。
- ・設計情報やプログラム・コードの作成によって、大幅な経費節減を可能にする。
- ・一連のチェックをコンピュータで行う為、品質の確保向上をはかれる。

6

リエンジニアリング

リエンジニアリングは自動化ツールの支援により、既存ソフトウェアシステムを調査したり、修正したりすることである。

リエンジニアリングの種類と目的

- | | |
|--|--|
| <ul style="list-style-type: none"> ・ 解析 ・ リストラクチャリング ・ リバースエンジニアリング ・ 移行 | <ul style="list-style-type: none"> ・ 既存システムを調べる。システムの品質を測定する ・ ソフトウェアの形式を変更する。 ・ システム・ドキュメントを再生し、設計情報を明らかにする ・ ある言語、ある環境へそれぞれ変換する。また新しい技術へ変換 |
|--|--|

必要性

- ・ ソフトウェアのバックログの増大
- ・ システム・ドキュメントが貧弱
- ・ システムの理解が困難であり変更が危険
- ・ 壊れやすいソフトウェア

効果

- ・ 保守しやすさの改善
- ・ 高い技術への移行
- ・ システム寿命の延長
- ・ CASEツールが支援するリポジトリ内への構成要素の取り込

リエンジニアリングの例

- ・リストラクチャリングツールを使用してプログラムを構造化した例



図 1.3 左側の FORTRAN のコードは十分な構造化がなされておらず、GO TO 文によるループがある。右側のコードは、Cobol Flow などのリストラクチャリング・ツール FOR-STRUCT により、標準 FORTRAN-77 に準拠して構造化されたもので、GO TO 文の数も減っている。

```

ADD-PROJ
IF PROJ-CODE AND NEW-PROJ NEXT
SENTENCE ELSE
GO TO ADD-PROG
IF INDX-P IS EQUAL TO 100 GO TO
WRITE-REC
MOVE EXP-REV-PROJ TO PROJ-TABL (INDX-P)
MOVE NO-PROG-OR-TE TO MODE-T (INDX-P)
ADD 1 TO INDX-P

```

図 1.3 構造化されていない COBOL コードの例。多数のエントリーポイントとイデントポイントがあり、プログラムの流れの GO TO 文も多量にある。

プログラムのリストラクチャリング・ツール

```

0990-RETOOL.
*PERFORM BY 0940-WRITE-GOOD-RECORD.
IF PROJ-CODE AND NEW-PROJ
IF INDX-P IS EQUAL TO 100
PERFORM 0990-WRITE-REC
ELSE
MOVE EXP-REV-PROJ TO PROJ-TABL (INDX-P)
MOVE NO-PROG-OR-TE TO MODE-T (INDX-P)
ADD 1 TO INDX-P
PERFORM 0990-ADD-PROG
PERFORM 0990-WRITE-REC

```

図 1.4 Computer Data Systems 社の Refact により、図 1.3 の COBOL コードを構造化したもので、GO TO 文も減らされている。

```

0940-SUPERSTRUCTURE.
*****
* PERFORMED BY 0930-ADD-PROJ -
*****
IF PROJ-CODE
AND NEW-PROJ
NEXT SENTENCE
ELSE
PERFORM 0990-ADD-PROG THRU 0990-END.
GO TO 0940-END.
IF INDX-P IS EQUAL TO 100
PERFORM 0990-WRITE-REC THRU 0990-END.
GO TO 0940-END
MOVE EXP-REV-PROJ TO PROJ-TABL (INDX-P)
MOVE NO-PROG-OR-TE TO MODE-T (INDX-P)
ADD 1 TO INDX-P
PERFORM 0990-ADD-PROG THRU 0990-END.
0990-END.
EXIT.

```

図 1.5 Computer Data Systems 社の Refact でリストラクチャリングされた COBOL コード。プログラムのイデントポイントの削減のようは顕著である。

- ・リバース・エンジニアリングツールを使用してプログラムからチャートを出した例

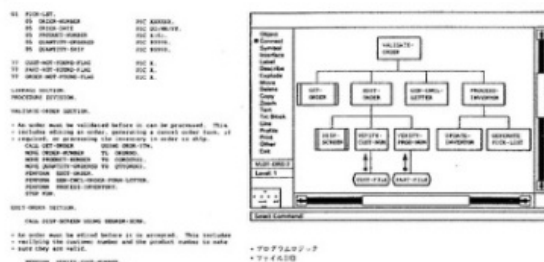


図 1.14 Intersys 社の Design Recovery により、COBOL ソースコードからストラクチャチャートを作成し、Excelerator フォントに置換したもの。

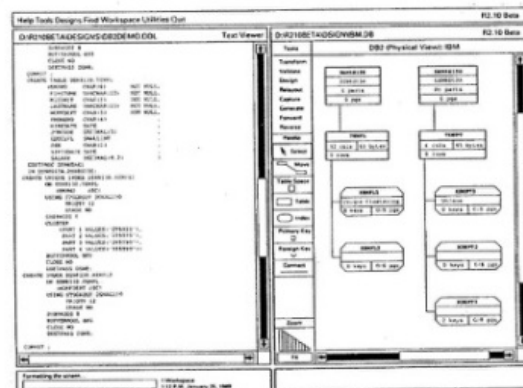


図 1.5 Backman Information Systems 社の Re-Engineering Product Set によって、両端を別の DBMS データベースの DDL が、両端を別のデータベース・デザイン・ツールにリエンジニアリングされている。

再利用可能性

システム開発にともなう生産物を、再利用する技術である

必要性

- ・ 開発の時間と費用を削減したい。
- ・ ソフトウェアの品質を向上させたい。
- ・ 生産性を向上させたい。

再利用の形態

- ・ プロトタイプ
- ・ データ
- ・ システムおよびプログラム構造の枠組み
- ・ データモデル
- ・ プログラムコードの一部
- ・ ソフトウェアパッケージ
- ・ ソフトウェア・ライフサイクル工程

再利用をソフトウェアの方法論およびツールに組み込む

10

人 材

システム開発における役割をよく理解し、それをうまくこなして
く特定のプレイヤーであり、高度の教育を受けた人材である。

必要性

優れた方法論や及びツールを効果的に使用し管理する為である

必要な人材

プロジェクト・マネージャ
システム・アナリスト
システム・エンジニア
テクニカル・エンジニア
リポジトリ管理者
カットオーバー・チーム
ユーザ・チーム
システム・チャンピオン 等

//

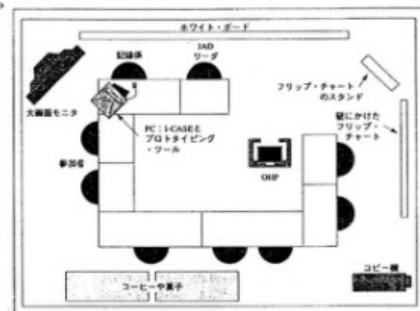
管 理

すべてのことを統合し運営する仕事

必要性

例えば サッカーのチームにおいても良い選手がいてもよい監督が必要（選手一人一人をコントロールできなければならない）

- ・ 開発全体にかかわる全体の要素を理解し、計画をたて実行し評価する。また評価にもとずき再度計画、実行する。
- ・ 開発チームに対して動機づけを与える。
- ・ 評価する場合の計量の指針を持つ。
- ・ 作業環境を整える。
- ・ ユーザ役割を明確にする。
- ・ 個々の責任を明確にする。
- ・ 官僚主義を避ける。 等



12